



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Case Study No.: 4 Name: Dhanish Ladwani PRN: 24070521020

1. Experiment Title:

Develop a web application using HTML and Javascript to validate a vehicle registration number.

2. Software/Tools Required:

Google Chrome (Web Browser), Visual Studio Code (Code Editor), Browser Developer Tools / Console, HTML, CSS, Javascript

3. Experiment Program Code:

Case Study 4.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vehicle Registration Number Validator</title>
  <link rel="stylesheet" href="Case Study 4.css">
</head>
<body>
  <div class="container">
    <div class="card">
      <h2>Vehicle Registration Validator</h2>
      <label for="regNo">Enter Vehicle Registration Number:</label>
      <input type="text" id="regNo" placeholder="e.g., MH31AB1234">
      <input type="button" value="Validate Registration" onclick="validateVehicleRegistration()">
      <div id="result"></div>
    </div>

    <footer style="text-align: center; margin-top: 30px; font-size: 14px; color: gray; border-top: 1px
solid lightgray; padding-top: 15px;">
      Made by Dhanish Ladwani PRN - 24070521020
    </footer>
  </div>

  <script src="Case Study 4.js"></script>
</body>
</html>
```

Case Study 4.css

```
body {
  font-family: Arial, Helvetica, sans-serif;
  background-color: whitesmoke;
  margin: 0;
  padding: 40px;
  display: flex;
  justify-content: center;
}

.card {
  background: white;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  max-width: 420px;
}
```

```

    width: 100%;
}

h2 {
    color: darkslategrey;
    margin-top: 0;
    margin-bottom: 20px;
    text-align: center;
}

label {
    font-weight: bold;
    display: block;
    margin-bottom: 8px;
    color: dimgrey;
}

input[type="text"] {
    width: 100%;
    padding: 10px;
    font-size: 15px;
    border: 1px solid silver;
    border-radius: 4px;
    box-sizing: border-box;
    margin-bottom: 15px;
    text-transform: uppercase;
}

input[type="button"] {
    width: 100%;
    padding: 10px;
    font-size: 15px;
    background-color: dodgerblue;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}

input[type="button"]:hover {
    background-color: royalblue;
}

#result {
    margin-top: 20px;
    padding: 12px;
    border-radius: 4px;
    font-weight: bold;
    text-align: center;
    display: none;
}

#result.valid {
    display: block;
    background-color: lightgreen;
    color: darkgreen;
    border: 1px solid green;
}

#result.invalid {
    display: block;
    background-color: lightpink;
    color: darkred;
    border: 1px solid red;
}

```

```

function isUpperAlpha(str) {
    for (let i = 0; i < str.length; i++) {
        const char = str[i];
        if (char < 'A' || char > 'Z') {
            return false;
        }
    }
    return true;
}

function isDigits(str) {
    for (let i = 0; i < str.length; i++) {
        const char = str[i];
        if (char < '0' || char > '9') {
            return false;
        }
    }
    return true;
}

function validateRegistrationNumber(regNo) {
    if (!regNo || regNo.trim() === "") {
        throw new Error("Registration number should not be empty.");
    }

    regNo = regNo.trim();

    if (regNo.length !== 10) {
        throw new Error("Length should be exactly 10 characters.");
    }

    const stateCode = regNo.substring(0, 2);
    const districtCode = regNo.substring(2, 4);
    const series = regNo.substring(4, 6);
    const vehicleNo = regNo.substring(6, 10);

    if (!isUpperAlpha(stateCode)) {
        throw new Error("First two characters must be uppercase alphabets (State Code).");
    }

    if (!isDigits(districtCode)) {
        throw new Error("Next two characters must be digits (District Code).");
    }

    if (!isUpperAlpha(series)) {
        throw new Error("Next two characters must be uppercase alphabets (Series).");
    }

    if (!isDigits(vehicleNo)) {
        throw new Error("Last four characters must be digits (Vehicle Number).");
    }

    return true;
}

function validateVehicleRegistration() {
    const inputElement = document.getElementById("regNo");
    const resultElement = document.getElementById("result");
    const regNo = inputElement.value;
    try {
        validateRegistrationNumber(regNo);
        resultElement.className = "valid";
        resultElement.innerHTML = `Valid: ${regNo.toUpperCase()} is a valid vehicle registration number.`;
    } catch (error) {
        resultElement.className = "invalid";
        resultElement.innerHTML = `Invalid: ${error.message}`;
    }
}

```

4. Output:

Vehicle Registration Number Validator

Enter Vehicle Registration Number:

Validate Registration

Valid: MH49CD7178 is a valid vehicle registration number.

Made by Dhanish Ladwani PRN - 24070521020

Vehicle Registration Number Validator

Enter Vehicle Registration Number:

Validate Registration

Invalid: Length should be exactly 10 characters.

Made by Dhanish Ladwani PRN - 24070521020

Vehicle Registration Number Validator

Enter Vehicle Registration Number:

Validate Registration

Invalid: Next two characters must be digits (District Code).

Made by Dhanish Ladwani PRN - 24070521020

5. Result/Conclusion:

A web application was developed to validate an Indian vehicle registration number of the format SSDDCCNNNN (e.g., MH31AB1234) using conditional statements and helper functions to check character ranges. The `validateRegistrationNumber()` function checks the total length, and verifies that the state code and series are uppercase alphabets while the district code and vehicle number are digits, using try-catch to raise and handle custom Error objects for each invalid case. The result — valid or invalid, along with the specific reason — is displayed dynamically on the page. This case study reinforced pattern-based string validation, custom exceptions and error handling in JavaScript.